

## 1. What is the main idea behind your distance 1 search?

The core idea of the distance search algorithm is to see each pattern index as a set of states to be processed separating those that have match distance 1 and match distance 0 into two distinct sets and maintaining the resulting sp, ep index for each pattern character that is “matched” by the BWT search.

That is for all indexes of the pattern we process the BWT search including all possible edits and maintain those that succeed (i.e  $ep > sp$ ) . Then upon the next pattern character we use the old results to process new results using dynamic programming to overwrite the old results with new using the old.

To maintain this the order of operations is most important, firstly we try ‘inserting’ so that we don't have to process the current  $i$  character of pattern, ( $pat[i]$ ). Following this for every recorded interval in our one distance and zero distance sets, we see what sp, ep intervals are after processing the  $pat[i]$  character. Those that succeed continue and we overwrite the set tables with these results. Following this to maintain all edit operations at each new pattern index, we add valid BWT searches for deleting, substituting and transpositioning via using the matched interval from the zero edit set.

The logic for each edit operation is as follows:

- **Exact Match:** Continues matching the current character  $pat[i]$ .
- **Deletion:** Decrements the pattern index without altering the suffix interval. That way we “delete” the character (by skipping over it)
- **Substitution:** Iterates over the entire alphabet, substituting the current pattern character and only continuing matching when a valid BWT sp, ep is found.
- **Insertion:** Again loop of the alphabet and check if we can finish the pattern with a new character, unlike substitution we do not skip an index since we are adding an additional character
- **Transposition:** Checks if swapping the current character and the preceding character forms a valid sp, ep this requires looking two indexes to the left of the pattern so we temporarily store the result and evaluate it when the  $pat[i]$  index is the same, as well as add a check to make sure there is a valid character to swap in the pattern.

An example on the given pattern = ‘abcd’ text = ‘abdcabxdbcycdzbabxcd’

## 2. What is the worst-case space and worst-case time complexity of your approach?

The worst case time complexity of the search phase of the algorithm is  $O(m^2 * |\beta| + K)$

Where  $K$  is the aggregate total of occurrences recorded, bounded by  $n$ ,  $m$  is the length of the pattern and  $|\beta|$  is the size of the unique alphabet in text.

Two cases give this time complexity, the base is when the entire pattern matches, in which we iterate over the pattern taking  $O(m)$ .

During the processing of the  $m$  distance-0 states, edit operations spawn new states with  $\text{match\_distance} = 1$ . At worst we create  $|\beta|$  since we loop over all possible valid characters in substitution and insertion.

Then for each state created from this we have to traverse the rest of the pattern looking for an exact match. The state can only propagate via exact matches for the remaining length of the pattern.

Each of the  $O(m * |\beta|)$  spawned states can move backward along the pattern at most  $m$  times.

The use of sets ensures that identical states  $(sp, ep)$  are evaluated only once per pattern character. So the maximum number of state evaluations in this phase is therefore bounded by  $O(m * (m * |\beta|))$ .

When the pattern is fully consumed the output is calculated via looping  $ep - sp + 1$  times to map suffix array indices to text positions for all passed states. Let  $K$  represent the total number of iterations, the worst-case scenario will take up to  $O(N)$  time.

(Assuming all sets and dictionaries are perfectly hashed and sized, else each insert takes  $O(m)$  or  $O(|\beta|)$  and the complexity becomes  $O(m^3 * |\beta|^2)$ ).

Thus the total complexity is  $O(m^2 * |\beta| + N)$  during searching.

The worst case space complexity of the search function is dominated by the `No_of_occurrence` table which is a 2D array of dictionaries that match for each character an array that notes `N_O_C[c][i]` the number of occurrences of the character  $c$  up to  $i$ . The size of  $C$  is bound by  $|\beta|$  and  $i$  is at most  $O(n)$  so the space complexity is defined as  $O(N * |\beta|)$ .

Outside of the search phase the total complexity is comprised of:

| Operation              | Time Complexity              | Space Complexity |
|------------------------|------------------------------|------------------|
| Generate BWT           | $O(n^2 \log n)$              | $O(n^2)$         |
| Build Occ              | $O(n *  \beta )$             | $O(n *  \beta )$ |
| Build First occurrence | $O( \beta  * \log( \beta ))$ | $O( \beta )$     |
| Compute suffix Array   | $O(n)$                       | $O(n)$           |